# BME 133 Lab 5

Welcome!

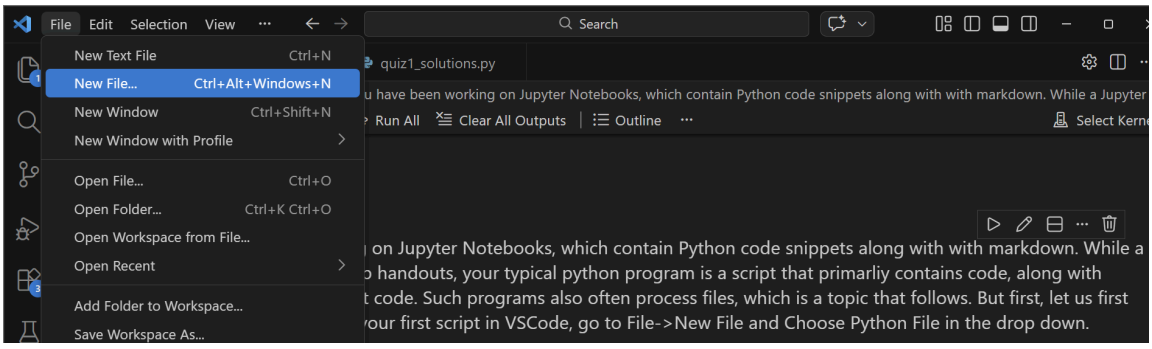
To begin, create a copy of this file using File -> Save as... and name the new file "Lab_5_LastName_FirstName.ipynb" using your last and first name. Make sure to execute all the code cells in the notebook and that the outputs show in your report. *Two points are allocated for reading and running the code blocks*. Before submitting, remember to add a collaboration statement in this mark down indicating the contribution of each member. You may edit the template below:

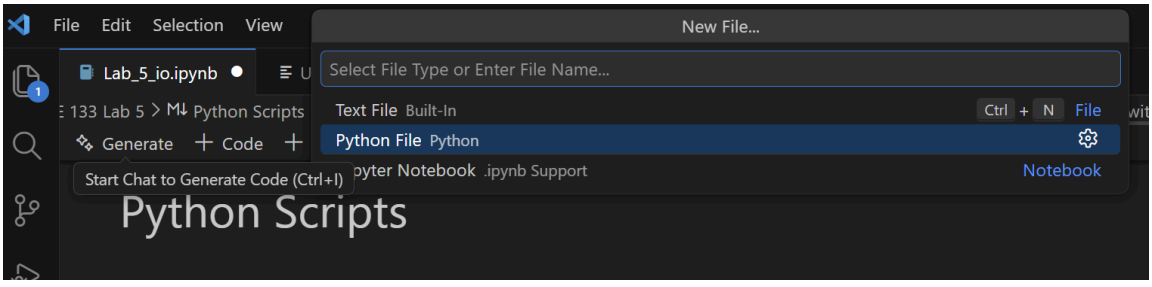
All three of us collaborated on the problems together

- Dania Abuirbaleh - 018756387
- Rohit Bhogal - 011221171
- Neha Raj - 018968170

## Python Scripts

Thus far, you have been working on Jupyter Notebooks, which contain Python code snippets along with with markdown. While a Jupyter Notebook is great for lab handouts, your typical python program is a script that primarliy contains code, along with perhaps comments to document code. Such programs also often process files, which is a topic that follows. But first, let us first create our first script. To create your first script in VSCode, go to File->New File and Choose Python File in the drop down as the following two screenshots show.



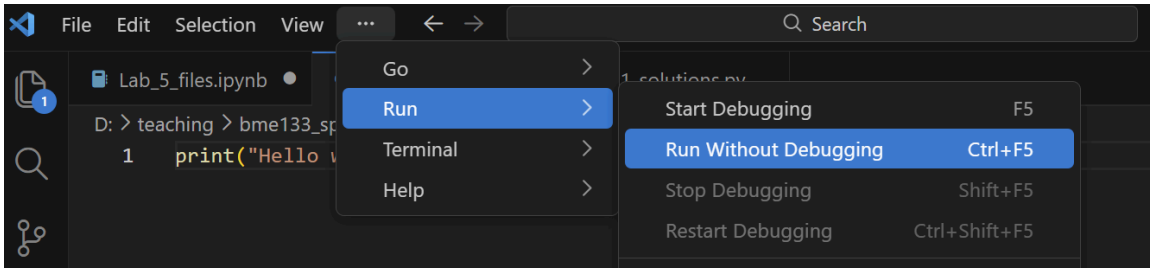



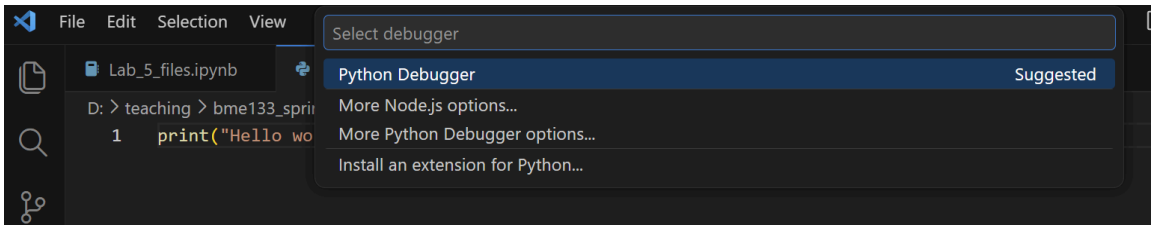
Then add the following code snippet to your file:

```
In [18]: print("Hello world from my first script")
```

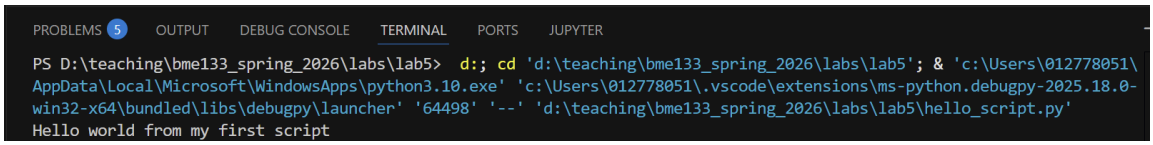
```
Hello world from my first script
```

Save your file as hello_script.py in the same folder where you have saved this notebook. Go to your script and run it by clicking Run->Run without Debugging from the menu and choosing Python Debugger in the prompt. See the two screenshots below.



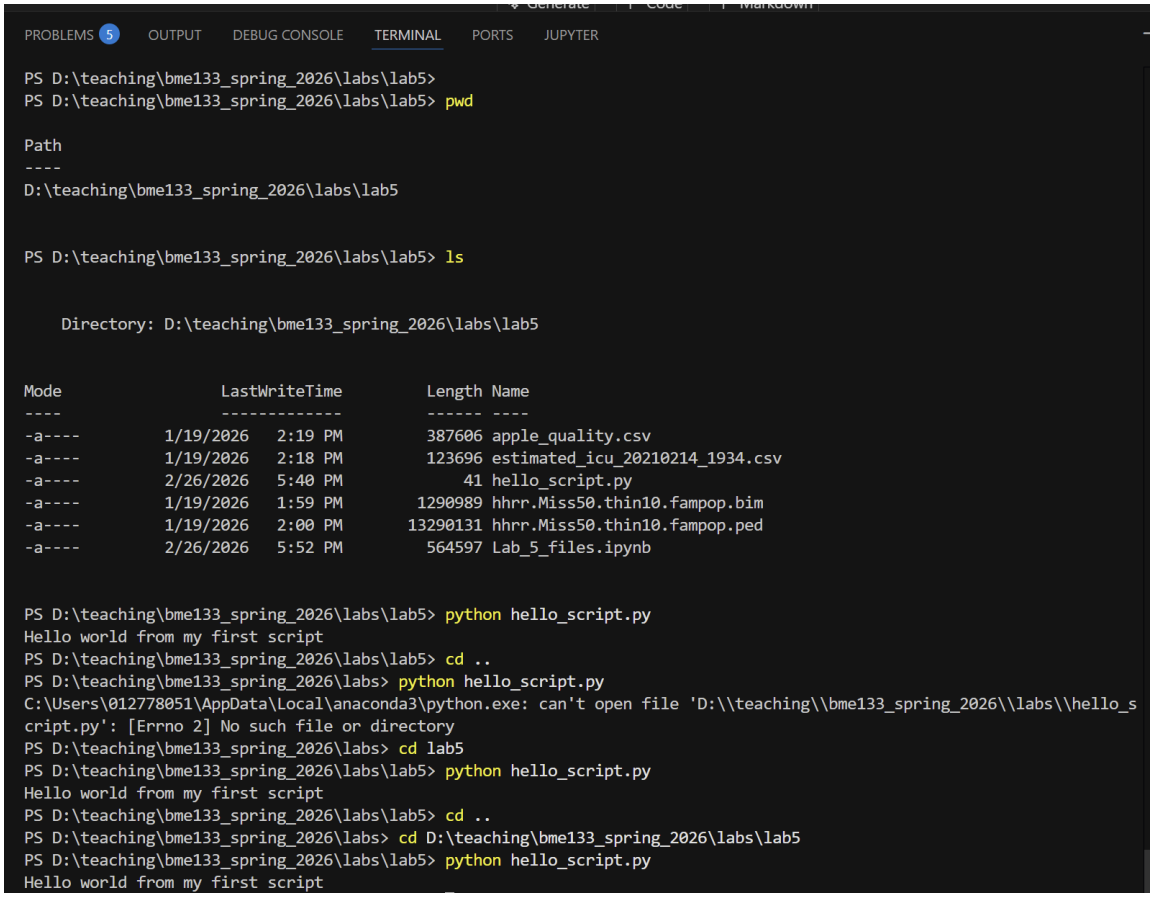




You will see your output under the terminal in VSCode as shown below.




VSCode is running a Python interpreter whenever you are running a code cell. But you can run the python interpreter to run scripts from other terminal environments. To run a script from a terminal enviroment, the interpreter needs to be able to find the script. If you call the interpreter from the location where your script is, it will find it in the current folder and run it. Let us show this with the VSCode terminal, but you should be able to do this in Anaconda Prompt as well. Under the terminal type `pwd` to check your current directory. The terminal outputs the *absolute path* of your current directory, where the drive is indicated by a label followed by a colon and the folders are separated by a forward or backward slash depending on your operating system. See screenshot below. You may also use `ls` to list the files and folders in your directory. If your hello_script.py is in the current folder, typing `python hello_script.py` will run the script and print the output. Otherwise, you will get an error. To test this, type `cd ..` in the terminal and type python `python hello_script.py`. The `cd` terminal command changes the current working directory and the two dots specify to go to the parent directory/folder. To go back to the folder that contains the script, we can use `cd lab5` where lab5, in my system, is the folder that contains the hello_script.py file. You can also pass the

absolute path that contains your script as shown in the next lines of the screenshot.




You can add all the statements you have learned thus far in scripts: control flow statements, functions, classes, etc.

## Reading and Writing Files

Almost always the data that you want to analyze is gathered from some external source that is not your Python program. We saw how we can use the built in input() function to get an interactive input from the command line. But that is limited. Often, biological data are available as files. Python has built-in functions to read and write files. Download the devices.txt file from Canvas to the same folder as this notebook if you have not done so already. Run the following code snippet and compare the output with the content in the file.

In [19]:
```
f = open("devices.txt")
print(f.read())
f.close()
```

```
FDA Product Code,Product Code Name,Regulation Number,Device,Regulatory Clas
s,Life Sustaining,Implant
BSK,CUFF,868.5750,2,Y,N
BSZ,GAS-MACHINE,868.5160,2,Y,N
```

The `open` method opens the file for reading and returns an object of the file class: it does not read the lines in the file yet. The read function reads the lines in the file. The close method closes the file so that it cannot be read from anymore. The readline below is used to read line by line:

In [20]:
```
f = open("devices.txt")
print(f.readline())
f.close()
```

```
FDA Product Code,Product Code Name,Regulation Number,Device,Regulatory Clas
s,Life Sustaining,Implant
```

If the file you are reading from is in another location than your script/notebook, you must specify the full path. Save a copy of devices.txt as devices2.txt in folder that is differnt from where this notebook is and try the command below. You should get an error.

In [21]:
```
f = open("devices2.txt")
print(f.read())
f.close()
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[21], line 1
----> 1 f = open(                )
      2 print(f.read())
      3 f.close()

File ~/anaconda3/lib/python3.13/site-packages/IPython/core/interactiveshell.
py:343, in _modified_open(file, *args, **kwargs)
    336 if file in {0, 1, 2}:
    337     raise ValueError(
    338         f"IPython won't let you open fd={file} by default "
    339         "as it is likely to crash IPython. If you know what you are
doing, "
    340         "you can use builtins' open."
    341     )
--> 343 return io_open(file, *args, **kwargs)

FileNotFoundError: [Errno 2] No such file or directory: 'devices2.txt'
```

I saved my devices2.txt in the folder D:\teaching\bme133_spring_2026\labs. So I can read it by specifying that path. Copy the path for your location and edit and run the code cell below.

In [8]:
```
f = open("/Users/rohit/repos/BME-133/exports/devices2.txt")
print(f.read())
f.close()
```

```
FDA Product Code,Product Code Name,Regulation Number,Device,Regulatory Clas
s,Life Sustaining,Implant
BSK,CUFF,868.5750,2,Y,N
BSZ,GAS-MACHINE,868.5160,2,Y,N
```

Note the double backslashes. A backslash inside a string is a special character. A backslash followed by some characters stand for specific characters. For instance a backslash followed by t ( `\t` ) stands for a tab. To interpret the backslash as a backslash, it needs to be escaped by preceding it with a backslash.

Opening and closing a file may be cumbersome and something easy to forget. The *with* construct allows you to read a file automatically opening and closing it as highlighted by the example below.

In [9]:
```
with open("devices.txt") as f:
    print(f.read())
```

```
FDA Product Code,Product Code Name,Regulation Number,Device,Regulatory Clas
s,Life Sustaining,Implant
BSK,CUFF,868.5750,2,Y,N
BSZ,GAS-MACHINE,868.5160,2,Y,N
```

To write to a file, we can use the write method.

In [10]:
```
with open("myfirstfile.txt", "w") as f:
    f.write("Hooray, wrote my first file!\n")
```

Open and check the written file in your file system. Note the `"w"` argument passed to the `open` method. It indicates that we are opening a file for writing. The `"w"` mode deletes any existing content.

In [11]:
```
with open("myfirstfile.txt", "w") as f:
    f.write("Oops, lost what I wrote :(\n")
```

To append content, you can use the `"a"` argument as follows.

In [12]:
```
with open("myfirstfile.txt", "a") as f:
    f.write("Appending a line to a file with content\n")
```

Python has a built in os module that has many built in functions to operate on files. For instance to check that the file above was created and remove it, you may use the following code construct.

In [14]:
```
import os
if os.path.exists("myfirstfile.txt"):
  os.remove("myfirstfile.txt")
else:
  print("The file does not exist")
```

```
The file does not exist
```

Run the code cell twice. Did you see what you expect? You can find other methods available from the os module at https://www.w3schools.com/python/module_os.asp

# Exercises

## Excercise 0

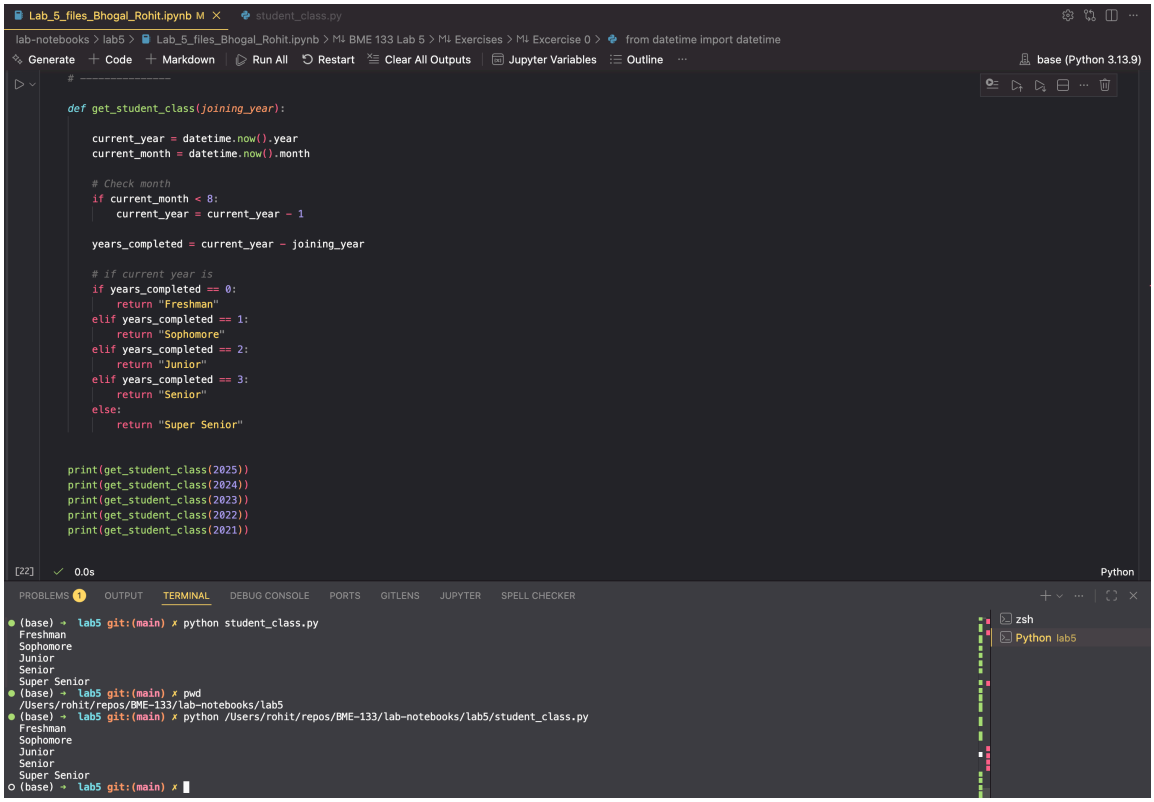
**(3 point)** Create a Python script called student_class.py that takes the year a student joined SJSU as an input and prints out "freshman", "sophomer", "junior", "senior" or "supersenior" based on the current year that the script is run (imagine the script could be run in the future: you may find the time, datetime or calendar python modules useful). Assume the student starts in the Fall of the year they specified. Run the script in both VSCode graphical user interface and the VSCode terminal/Anaconda Prompt.

In [ ]:
```python
from datetime import datetime


"""
Edge case
---------

If it's 2027 and joining_year = 2026
but current_month < 8 it means they're a freshman still not a junior

2027 - 2026 = 1 -> 'Sophomore" Not TRUE. They're still freshman. Subtract 1
"""

# If else version
# ---------------

def get_student_class(joining_year):

    current_year = datetime.now().year
    current_month = datetime.now().month

    # Check month
    if current_month < 8:
        current_year = current_year - 1

    years_completed = current_year - joining_year

    # if current year is
    if years_completed == 0:
        return "Freshman"
    elif years_completed == 1:
        return "Sophomore"
    elif years_completed == 2:
        return "Junior"
    elif years_completed == 3:
        return "Senior"
    else:
        return "Super Senior"
```

```
print(get_student_class(2025))
print(get_student_class(2024))
print(get_student_class(2023))
print(get_student_class(2022))
print(get_student_class(2021))


# Lookup table version
# --------------------
# def get_student_class(joining_year):

#     current_year = datetime.now().year
#     current_month = datetime.now().month
#     class_names = ['Freshman', 'Sophomore', 'Junior', 'Senior']

#     # Check month
#     if current_month < 8:
#         current_year = current_year - 1


#     years_completed = current_year - joining_year

#     if years_completed < 0:
#         return "Not enrolled"
#     if years_completed >= len(class_names):
#         return "Super Senior"
#     return class_names[years_completed]

# print(get_student_class(2025))
# print(get_student_class(2024))
# print(get_student_class(2023))
# print(get_student_class(2022))
# print(get_student_class(2021))
```

```
Freshman
Sophomore
Junior
Senior
Super Senior
```

Attach the screenshots of you running the script in the GUI and the terminal by pasting them in this markdown.




# Excercise 1

**(5 points)**

Create a class called Device whose member variables correspond to the fields in https://www.fda.gov/media/87739/download.

Add a constructor (__init__) as appropriate.

The class should also have a method called is_implant that returns true if an object instance has its implant field as Y and returns false if it has an implant field of N.

Create a list of Device objects and read the two devices in the devices.txt file into this list of objects.

Test your is_implant method with these two devices.

Add a third Device object to the list with the information in the third row (BTL) of the file linked above.

Write the list of three objects to a new file called three_devices.txt.

The new file should have the same format as the devices.txt. You may find the strip() and split() methods of the string class useful.

In [ ]:
```
# create a class named "Device"
# method is_implant return true if object instance has it's implant field as
```

```
class Device:
    def __init__(self, fda_product_code, product_code_name, regulation_numbe
        self.fda_product_code = fda_product_code
        self.product_code_name = product_code_name
        self.regulation_number = regulation_number
        self.device_regulatory_class = device_regulatory_class
        self.life_sustaining = life_sustaining
        self.implant = implant

    def is_implant(self):
        if self.implant == 'Y':
            return True
        if self.implant == 'N':
            return False


# read file from 'devices.txt'
# create a list [] of device objects and read


# open file
# read the file -> returns list of strings (including header)
f = open('devices.txt', 'r')
header = f.readline() # reads header, advances cursor
init_devices = f.readlines() # rest of the lines after header
f.close()

devices = [] # list of Device objects

# loop list of init devices
for device in init_devices:
    # create a Device
    # remove newline, split by comma
    device_fields = device.strip().split(',')

    # spread the list of device fields into the params of Device class
    # append new device to the devices list
    new_device = Device(
        fda_product_code=device_fields[0],
        product_code_name=device_fields[1],
        regulation_number=device_fields[2],
        device_regulatory_class=device_fields[3],
        life_sustaining=device_fields[4],
        implant=device_fields[5],
    )

    # NOTE: alternatively use unpacking '*'
    # new_device = Device(*device_fields)

    devices.append(new_device)
    # test device
    print(new_device.is_implant())

# if testing outside of function
print('(Testing outside function)')
```

```
for device in devices:
    print(device.is_implant())

# add third device
devices.append(Device(
    'BTL', 'VENTILATOR', '868.5925', '2','Y','N'
))

# create a new file 'three_devices.txt'
with open('three_devices.txt', 'w') as f:
    f.write(f"{header}") # headers
    for device in devices: # device is an object
        f.write(f"{device.fda_product_code},{device.product_code_name},{devi
```

```
False
False
(Testing outside function)
False
False
```